The Software Equation

It is a dynamically multivariant Model that assumes a specific distribution of effort over the lifecycle of a slw development project. It's Empirical in Nature. It is given by,

$$E = \left\{ \frac{L.C * \beta^{1/3}}{P} \right\}^3 * \left[\frac{1}{t^4} \right]$$

where, E is the effort in person-Months (or) person-years, t = time (or) project duration in months / years

β = special skills factor

P = productivity parameters

⇒ β = 0.16 for small programs
= 0.39 for > 70KLoC (inline documentation inside it also present)

⇒ P reflects the overall process Maturity. It is measured using:

1) How much are good Slw Engineering techniques are used?
2) The level of prog languages
3) The state of Slw Environment (starting from scratch or not)
4) Skills and Experience of slw Team
5) complexity of Application

⇒ P is usually 2000 for real-time embedded software
⇒ 10,000 for telecom and System software
⇒ 28,000 for Business Applications.

⇒ the lesser value of P, more will be the effort
⇒ Accordingly E, t (effort (#people), time taken) is calculated
⇒ where will slw equation fail?
as LoC is not good parameter to consider always
2 languages efforts cannot be compared.
⇒ careful about years, months in the units for the slw eqn formula.

⇒ Make/Buy decision
(few changes to already off-the-shelf slw)
① ⇒ slw may be purchased off-the-shelf
② ⇒ slw may be Modified and integrated to meet specific Needs
③ ⇒ slw may be custom made by a third party (outsourcing)
(Always needn't to develop slw from scratch)
eg: Github

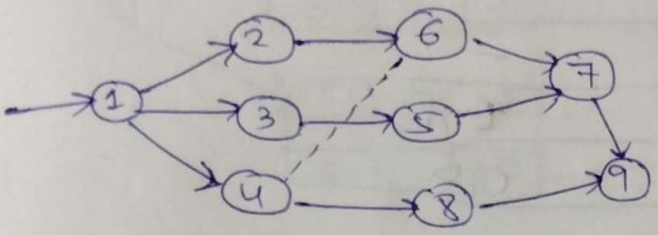
⇒ Project Scheduling
⇒ We need to track progress of developers in a slw project

⇒ ① PERT - Project Evaluation and Review Technique
→ It displays task sequences required for a project
→ It contains sequence of Activities from the Beginning to the End of the project
→ It also displays inter Relationships/Dependencies blw tasks.

→ There are 3 values that are used: Min, Avg, Max (Associated with completion of each task)

→ An arrow (⟶) shows the Actual task.

→ It makes it possible for Managers to summasize more detailed task schedules and track sub tasks leading to the completion of a Major Activity in a higher level PERT chart.



each ⟶ is a Milestone. and one of min, Avg, Max will be these

···· → to show Interrelationships

① → ②
① → ③
① → ④
} pasalled computations

Inorder to get Approx time for whole project,

Effort formula $E = \frac{(min + 4.avg + max)}{6}$

↓ Avg Effort

(Bellshaped kind of graph)

→ complete project per task effort (milestone) (sum of all of them will give complete effort)

⇒ If we give max for every milestone and if we have a path where we get max value. Then if any thing goes wrong in that path then things are vulnerable.

⇒ that is called critical path (CPM-Critical Path Method)

⇒ If we are doing a project, then we have to focus on critical path so that nothing goes wrong there.

⇒

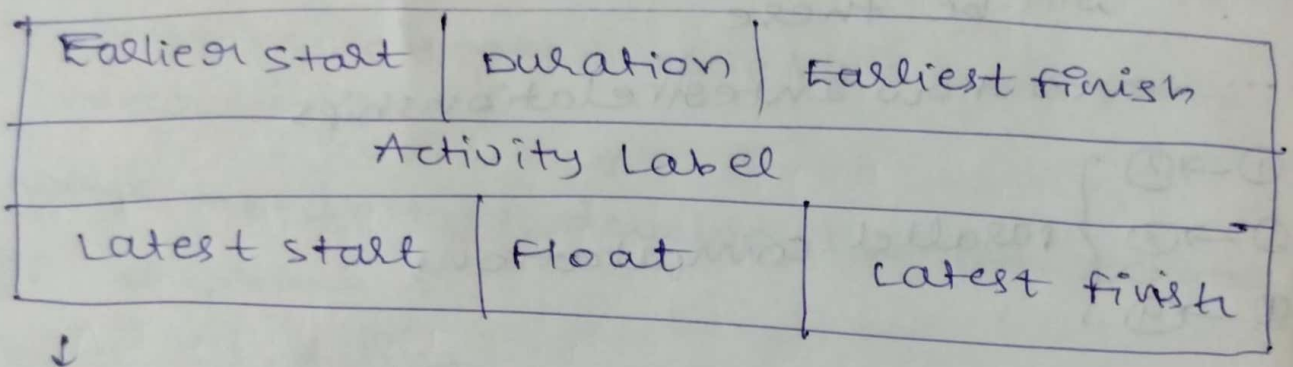
Activity	Duration	Predecessor
A	6	–
B	4	–
C	3	A
D	4	B
E	3	B
F	10	–
G	3	E, F
H	2	C, D

PERT CHART

Generation from data given

→ Data given

⇒ Node Representation



↓

This should be done for each Activity

⇒ Earliest start is the date/time at which activity can be started at the earliest

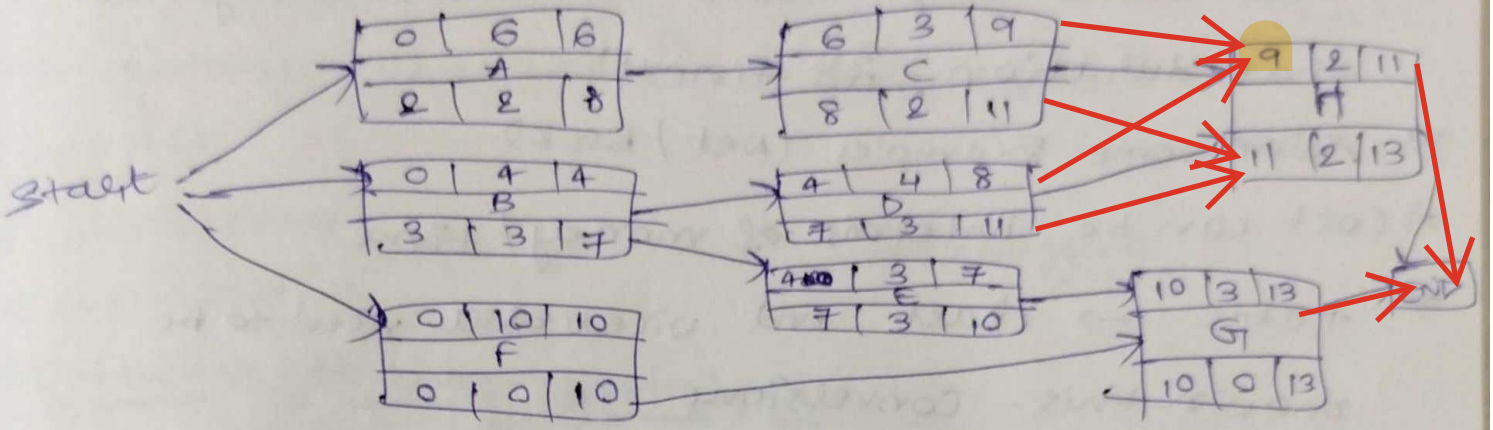
⇒ Earliest finish is the date/time at which activity can be completed at the earliest

⇒ Latest start is the date/time at which activity can be started as late as possible

→ Latest finish is the date/time at which the activity can be completed at latest

→ float = the difference b/w earliest start and latest start (or) earliest finish and latest finish

→



→ ⇒ Based on predecessor

① earliest start → based on dependencies (left to right)
we take max of all.
as we need to finish all predecessor.
max(earliest finish)

② earliest finish

③ latest finish
(right to left)
The nodes before END, check the highest earliest finish and it will be the latest finish of all that nodes.
max(11, 13) = 13

For previous layer nodes, we see Dependencies and we decrement duration from latest finish and max of dependencies (if any) max((10-3), (13-3)) = 10

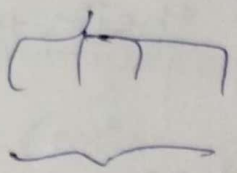
~~behaviour~~ →

→ structure → hierarchy

→ Not mandatory to make all diagrams

→ NO source code is written

→ Behaviour diagram



Things in Action

Small team → UML specialists } for project → PERT charts

→ Inventary checks where the goods are

→ project

~~project~~

→ consistency

→ constraints

Debugging

→ Software testing strategies

→ We generate a source code and that source code has to be tested to uncover and correct as many errors as possible before deployment to the uses Base.

→ The goal here is to design a series of testcases that have high likelihood of finding errors.

→ This will provide a series of test for

① Exercising Internal logic of slw components

② " I/o domains of program to uncover Errors in program function behaviour and performance

eg:- screen fitting for canvas, etc

→ (i) Glass Box / White Box Testing } 2 Testing Techniques

→ (ii) Black Box Testing / Behavioral testing

white box testing

→ (i) The slw Engineer derives test cases that

(or)

slw Testing strategies

① Guarentee that all Independent paths within a module have been exercised atleast once. (every LoC executed atleast once)

② Exercise all logical decisions on their true and false sides

③ Execute all loops at their boundaries and within their operational Bounds.
(print that it reached Bounds inside loop (if else statement)

④ Exercise all Internal data structures to ensure their validity

(ii) It enables the Engineer to derive a set of Input conditions that will fully Exercise all functional Requirements for a program

→ It finds errors of the following categories

① Incorrect (or) Missing functions
(Print statements at start, end of the function)

② Interface Errors
(datatypes used, Etc)

Q1. Can we go inside loop directly from outside

Q2. Do we need to do this?

Q3. under what circumstances it is possible to do it?

Q4. Why (or) where do you need RedBlack tree?

Q5. Look into Splay tree (Inherently Balanced)

Q6,
→ ① What is data structure
→ ② why do you need data structure?
→ ③ Optimizations inherent to data structure? which cases yes, which cases No?

② Yes, No

③ Not optimal for certain, optimal for certain Applications

① entity which can perform specific work with the functions declared for it

③ Errors in data structures (or) external database access

④ Behavior (or) performance errors (system changes for execution, lag time, etc)

⑤ Initialization and termination errors

Eg:- (exit codes Incorrect, (deallocating the extra space you created, close the file) Initialization error will n't happen. (do it next line when you allocate, opened file) → Memory Management)

⇒ This type of testing is done during later stages of program development

⇒ BBT → focuses on performance (Module testing)

WBT → Code Testing

⇒ 3 techniques for I

⇒ Integration Testing for S/w

⇒ once the modules have been tested Individually, will they work after they are Integrated?

⇒ 1, Smoke Testing

⇒ There are several Activities.

(i) S/w components that are completely coded are made into a Build.

A Build consists of all data files, library routines, reusable modules for a given product function. (dynamic)

(ii) A series of tests is designed to expose errors

(iii) This Build is integrated with other Builds and then smoke tested daily.

Subgroups
leaders
Core team
Technologies used
work/Module division
Google Doc → Technical problems faced
Excel → per day work

P:D ratio
↓ ↳ Defenders
rests
(1:1)

⇒ The Integration is normally done bottom-up
⇒ This testing strategy is used in time critical projects

⇒ 2) Regression Testing

⇒ Each time a new Module is added to the project. It may cause problems to occour with functions that previously worked perfectly
⇒ Regression testing is the re execution of some subsets of tests that have been conducted to ensure that "changes have not propagated unintended errors into system".

Set of testcases for each subgroup

⇒ Mistake proofing of s/w
⇒ (Shigeo Shingo) (japanese, Bomin 1909 - 1990)
⇒ Worked on Automobiles (Poka-yoke)
⇒ ① Manufacting Automobiles should have Min Defects
⇒ ② SMED (Single Minute Exchange of Dies)
⇒ ③ JIT (just in time) production strategy
⇒ 1, 2, 3 → contributions of him
⇒ Toyota → worked in it
⇒ In 1960's, he developed a Quality Assurance System called Poka-yoke which had the following charecteristics
(i) prevention of a potential Quality problem before it occurs

20-30 testcases for the functions you write

Smoke testing in a car garage

Subgroup work, deadline

(ii) Rapid detection of quality problems if they are Introduced

(Introduce a bug, how quickly it returns Back that there is a problem before damaging)

QAS:

⇒ The Detailed characteristics are as follows:-

1, It should be simple & cheap.

(It should be cost effective)

2, It should be part of the process

(It should be Integrated into an Engineering Activity)

3, It should be located near the process task where the Mistakes can occur.

(Rapid Feedback)

⇒ Initially it was made for 0 Quality control (0 defects)

⇒ In slw, they are usually small pieces of code (less than 100 lines long) and they are easy to run

(Exception Coding (where to keep these statements is the point to look into))

⇒ SAD: System Analysis and Design

⇒ System Engineering

⇒ It consists of a set/Arrangement of elements that are organised to Ecomplish some predetermined goal by processing Information.

⇒ The goal would normally be to support Bussiness function (or) develop a pdt that can be sold to generate Business Revenue

⇒ Elements: (Need to Maintain slw)

① slw → programs, data structures, procedures, constraints, Etc

② Hlw → These will consist of electronic devices (Eg:- sensors), switches, tele communication devices, Etc that provide external world function.

SHPPDD
(shippedd)

③ people → The users and operators of Hlw and slw.

④ Database → It is a large organised collection of info that is accessed via software.

⑤ Documentation → Descriptive Information (Hardcopy Manuals) online help, websites that explain the use and operation of system

⑥ procedures → The steps that define the specific use of each system element (or) the procedural context in which system resides

⇒ System Engineering is a collection of top-down (or) Bottom-up methods for proper Identification of the system.

This begins usually with the world view
The entire business/pdt domain is examin-ed to ensure that proper Business/ technolgy context can be established
Initially, a Broad context is ~~Established~~ established. subsequently, detailed technical Activities are outlined.

$WV = \{D_1, D_2, \cdots, D_n\}$
(WV: world view; $D_1, D_2, \cdots, D_n$: set of domains)

Each domain has specific Elements (E_j) Each of which serves some role in Ecomplishing the Objectives of the domain (or) components. i-e; $D_i = \{E_1, E_2, \cdots, E_m\}$
Finally each Element is Implemented by specifying technical components C_k that acheive the Necessary function for an Element i-e; $E_j = \{C_1, C_2, \cdots, C_k\}$

⇒ The system narrows the focus of work as he/she moves down in the hierarchy

↓
Friday Eve 5pm deadline to Document this

⇒① System Engineering is a Modeling process It helps in defining the processes that serves the need of the view until consideration

⇒② Represents the Behavior of processes and Assumptions on which it is Based.

⇒③ It explicitely defines both exogenous Hexogeneous (link one constituent of each view to another at the

?????

same level) and Endogeneous (Link individual components of a constituent at a particular view) input to the Model

→ Represent all linkages (Including output) to better understand the view.

⇒ System Simulation

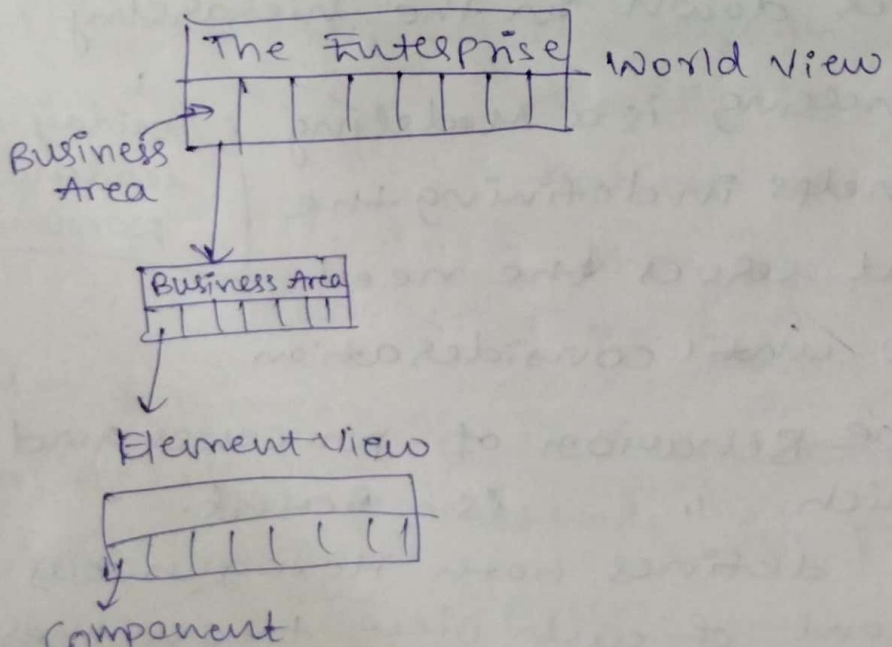
⇒ Normally we built systems like the Wright Brothers built airplanes 'Built the whole thing, push it off the cliff, let it crash and start all over again' (Knowledge is gained) (Ride becomes better and better) → Reactive system

⇒ Imitation of flight of 1st flight by Wright Brothers? Ans: 12 seconds → check this!!

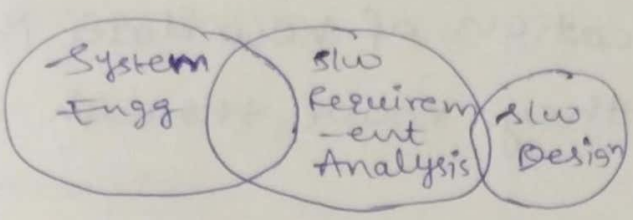
⇒ We do this today like Adventure Flight. But, we can use simulation tools to test drive the software system before deployment.

⇒ Business process Engineering, Application Architecture Product Engineering → Read in pressmen ← Requirement Engineering System components Engi

⇒ Complete layout of Sw system



System Analysis



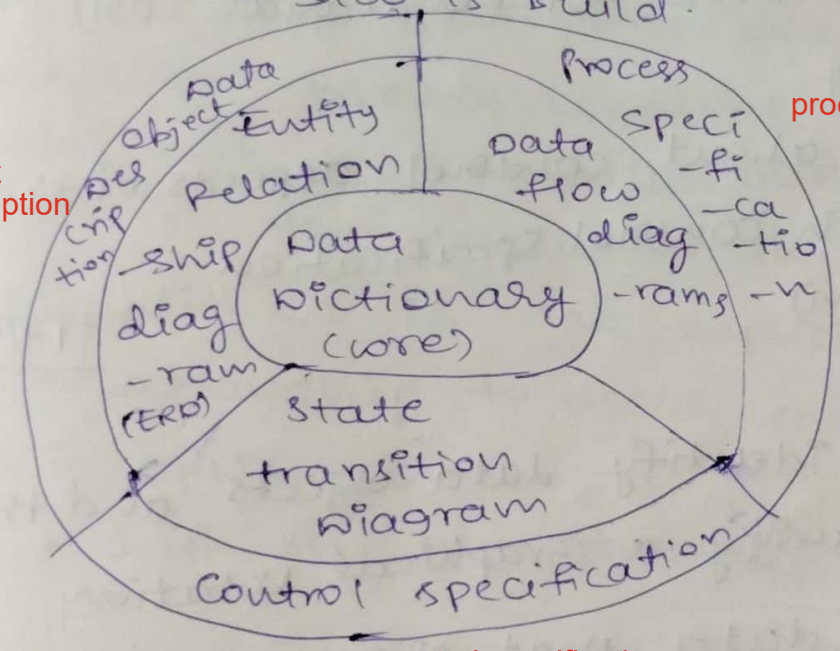
→ We do SW Requirement Analysis in detail :- (5 steps)

1) Problem Recognition
2) Evaluation and Synthesis
3) Modeling
4) specification
5) Review

PESMSR

→ These are 3 primary objectives :-

1) To describe formally what customer Requires
2) Establish a Basis for the Creation of a SW Design
3) To define set of Requirements that can be validated once the SW is Build.



data object description

process specification

control specification

→ provide these documents for your project

→ The Core is the Data Dictionary which contains Description of all data objects consumed/ produced by the SW.

→ ER Diagrams depict Relationships b/w data objects.

⇒ The data flow diagrams serve 2 purposes

* To provide an Indication of how data is transformed as ~~we~~ they move through the system.
* To depicts the functions and subfunctions that transforms in the data flow.

⇒ A description of each function presented in DFD is contained in the process specification

⇒ state transition diagram shows how the system Behaves as a consequence of external events.

⇒ It represents various Modes of Behaviour ~~(states)~~ (called states of system) and the manner in which transitions are made from state to State.

⇒ ~~It~~ serves as the Basis for Behavarial (or) Blackbox testing.

⇒ Additional info about control aspects of slw is contained in control specification.

⇒ ER Diagrams 19/2012

⇒ It is used ~~to~~ identify data objects and their relationships using a Graphical Notation

⇒ It defines all data that are entered, stored, transformed and produced within an Application.

⇒ ~~It~~ Represents data network that exists for a given System

⇒ Data Model has 3 inter related pieces of Information.

① Data object.
② Attributes
③ Relationships
↓
that connect data objects to one another.

⇒ The data object is a Representation of almost any composite information that must be understood by sw.

⇒ Also, 2 more criteria is there along with Above 3.

④ Cardinality
⑤ Modality

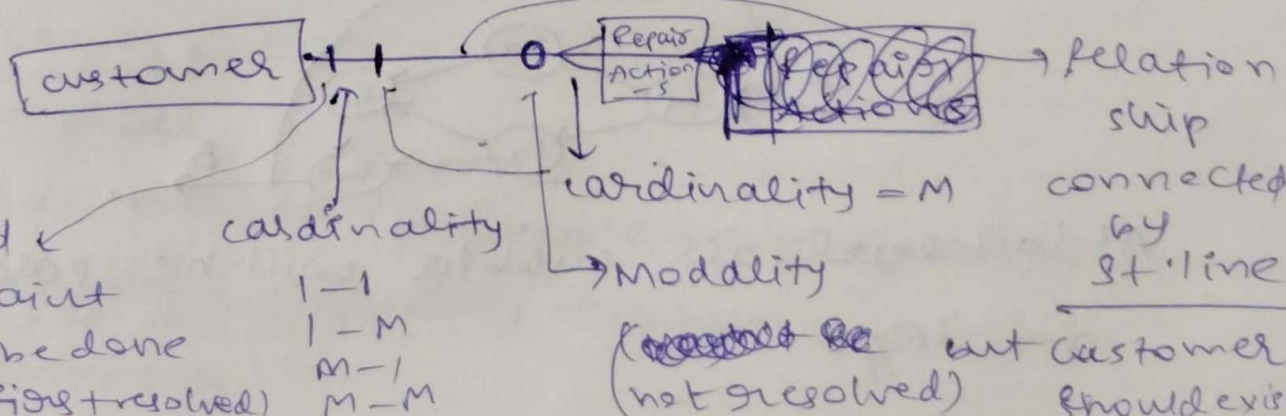
⇒ cardinality means how many Instances of object X relate to how many occurances of object Y. (one-one, one-many, many-one, many-many)

⇒ Modality means whether (or) not a particular object must participate in the relationship.

Modality = 0 if no explicit need for the relationship to occur (or) relationship is optional.

= 1 if the relationship is Mandatory.

Eg:-



⇒ DFD (data flow diagram)

⇒ It is a graphical Representation that depicts Information flow and the transforms that are applied as data move from Input to output.

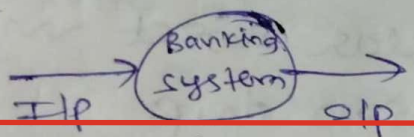
⇒ The Basic form of DFD is called data flow Graph (or) a Bubble chart.

⇒ It is used to Represent a system (or) s/w at any level of Abstraction.

⇒ DFD Level 0 :-

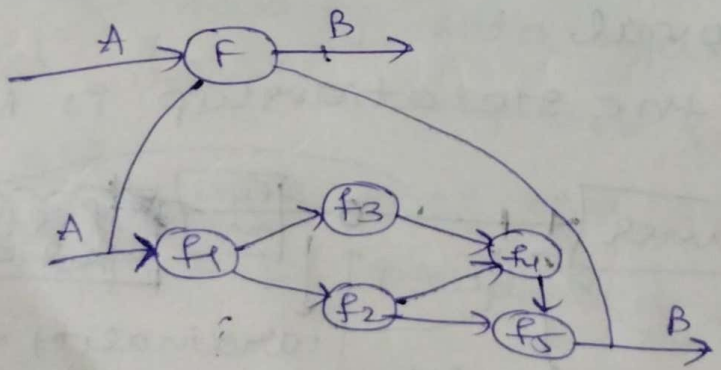
⇒ It is called fundamental system model/ context Model and it represents entire s/w element as a single Bubble with an Input, Output Arrow Indicating I/p and o/p.

Eg :-



⇒ DFD level 1 :-

Eg :-



At last level, all bubbles will be paragraph of code//.

⇒ ER Diagrams → Tells static part
DFD " → Tells Dynamic part
} Needed for project

⇒ Data Dictionary

⇒ It consists Quasyformal Grammer for describing contents of objects defined during Structural Analysis.

Document Diff blw what done and what is o/p

⇒ It contains

1) Name
2) Alias (Definitions → checks)
3) Where used / How used
4) content Description
5) supplementary Information

⇒ card storz → Mech
kuka → Robotics } company

⇒ Standards for SW : ISO standards (Company Standards)

⇒ ISO 9000. Quality standards

⇒ It describes the elements of a quality assurance system is

In general terms:

These Include:

1) Organisational structure
2) procedure
3) processes
4) Resources

which are needed to Implement quality planning, Quality Control, Quality Assurance, Quality Improvement.

⇒ 1, Organisional structure: 2-3 pgs of doc containing employees details of their work.

⇒ 4, Resources → Environment friendly

⇒ 2, procedures → clear, understandable, user friendly (office procedures)

⇒ 3, processes → Technical processes (sequence of actions done correctly)

⇒ certification is given for only 1 year. (can renew after it)
↓for
(ISO 9000 series compliant)

⇒ How this is to be implemented is not described in a document.

⇒ 180 9001 Standard

⇒ This applies to slw Engineering and consists of 20 Requirements that must be present for an effective Quality Assurance system

⇒ ISO 9001 applies to all Engineering diciplines A special set of guidelines which are detailed in ISO 9000-3 have been formulated to help Interpret standard for use in slw process.

UML	Simple development (just do what needed)	consistency validity	# Enthusiasm is Impinswe (systematic Approach is the soln)